

FashionOS

Complete Project Documentation

Version: 1.0 (Draft)

Owner: Project Manager

Last Updated: 5 October 2026

Status: For team review

Updated to match FashionOS SRS v1.0 (final). FashionOS is the client's own business: one shop and one website that share one stock, with khata (credit), WhatsApp bills, Tally integration, employee management and saree fall-pico. The earlier marketplace / SaaS plan is removed. The SRS says *what* to build (requirement IDs like **FR-KHT-02**); this document says *how* to build and run it. Anything not confirmed is marked *[Assumption]* or *[TBD]*.

Contents

What Changed in v1.0	2
1 How to Use This Document	2
1.1 Open Decisions That Change the Build	2
2 Product Overview	3
2.1 Vision	3
2.2 Goals	3
2.3 Users	3
2.4 Feature Map by Phase	4
2.5 Success Metrics [<i>Assumption</i>] (targets to confirm)	4
3 Scope and Roadmap	4
3.1 In Scope (Phase 1)	4
3.2 Out of Scope (for now)	5
3.3 Roadmap [<i>Assumption</i>] (to be re-estimated)	5
3.4 Milestones	5
3.5 Rollout Order	5
4 System Architecture	6
4.1 High-Level Diagram	6
4.2 Components	6
4.3 Key Design Decisions	6
4.4 Flow in Mode B (FashionOS POS): Khata Sale to WhatsApp and Tally	7
4.5 Flow in Mode A (counter bills in TallyPrime; first step of rollout)	7
4.6 Environments	8
5 Module Technical Notes	8
5.1 Shared Stock and Reservations (FR-CAT-02..04)	8
5.2 Offline POS (FR-POS-08)	8
5.3 Khata Ledger (FR-KHT)	8
5.4 WhatsApp Pipeline (FR-WAP)	9
5.5 Online Payments and Pay Now Links (FR-PAY, FR-PRT)	9
5.6 Tally Integration (FR-TLY)	9
5.7 Employees and Payroll (FR-EMP)	10
5.8 Fall-Pico (FR-FPC)	10
6 AI Integration	11
6.1 AI Use Cases	11
6.2 AI Service Rules	11
6.3 Input and Output Examples	11

6.4	Models and Cost [<i>Assumption</i>]	12
6.5	Prompt Management	12
6.6	Fallbacks	13
6.7	Data Privacy for AI	13
6.8	AI Quality Testing	13
7	Database Schema	13
7.1	Main Tables	13
7.2	Relationships (simple view)	14
7.3	Database Rules	14
8	API Documentation	15
8.1	Standard Response Format	15
8.2	Common Error Codes	15
8.3	Endpoints	15
8.4	Sample: Push a Khata Bill from POS	16
9	UI/UX Guidelines	17
9.1	Interfaces	17
9.2	Sitemap	17
9.3	Design System [<i>Assumption</i>] (designer to finalise)	18
9.4	Key User Flows	18
9.5	Design Deliverables	18
10	Team Roles and Sprint Plan	19
10.1	Roles [<i>Assumption</i>] (5 members)	19
10.2	Sprint Plan (2-week sprints) [<i>Assumption</i>] (to be re-estimated after SRS sign-off)	19
10.3	Task Board Rules	20
11	Developer Guide	20
11.1	Repository Structure	20
11.2	Coding Standards	21
11.3	Git Workflow	21
11.4	Environment Setup (new developer, target 30 minutes)	21
11.5	Environment Variables	22
12	Security and Performance Checklist	22
12.1	Security	22
12.2	Performance	23
12.3	SEO Checklist (website)	23
13	Test Plan	24
13.1	Test Types and Tools	24
13.2	Core Test Cases (examples; full list tracked in the test tool)	24

13.3 Bug Severity	25
13.4 Release Gate (must be true to go live)	25
14 Deployment and Go-Live Guide	26
14.1 Release Steps (Production)	26
14.2 Installing the Tally Sync Agent (Cloud Tally Server)	26
14.3 Data Migration and Parallel Run	26
14.4 Rollback and Contingency	27
14.5 Monitoring and Alerts	27
14.6 Go-Live Checklist	27
15 Staff Handover and Training Manual	28
15.1 What You Receive	28
15.2 Daily Tasks	28
15.3 Things to Remember	29
15.4 Frequently Asked Questions	29
15.5 Support and Maintenance	30
15.6 Training Plan	30
16 Risks, Assumptions and Change Log	30
16.1 Risks	30
16.2 Key Assumptions	31
16.3 Change Request Process	31
16.4 Change Request Log	32
16.5 Document Version History	32

What Changed in v1.0

1. The technology stack is decided: FastAPI, React + Vite + TypeScript, PostgreSQL, Celery + Redis (earlier text about NestJS, Next.js and BullMQ is removed).
2. Tally data moves to a cloud server in India that runs 24 hours, with the sync agent on that same server.
3. The shop bills in TallyPrime today with one counter, so FashionOS starts in **Mode A** (it imports from Tally) and the counter moves to the FashionOS POS later (**Mode B**) after a clean parallel run. The roadmap, milestones, sprint plan, rollout order, Tally design, setup, rollback and risks are updated for this.
4. The bill number format will change; a new series starts at cutover.
5. GSTIN is confirmed.

1. How to Use This Document

Section	Main reader
1. Product Overview, 2. Scope and Roadmap	Client + Team
3. Architecture, 4. Module Technical Notes	Developers
5. AI Integration, 6. Database, 7. API	Backend, Integration, AI engineers
8. UI/UX, 9. Team and Sprints	Designer, PM, all
10. Developer Guide, 11. Security	Developers
12. Test Plan, 13. Deployment and Go-Live	QA, DevOps, Accountant
14. Staff Handover and Training Manual	Client and shop staff
15. Risks and Change Log	PM, Client

Rule: one source of truth. Requirements live in the SRS. If something changes, change the SRS first, then this document, then the code.

1.1 Open Decisions That Change the Build

#	Decision	Current assumption	Build impact
1	Where is the shop bill made: FashionOS POS or Tally?	Start in Mode A: the counter keeps billing in TallyPrime and FashionOS imports it. Move to the FashionOS POS (Mode B) after a clean parallel run. Confirmed: billing is in TallyPrime today	Decides the direction and volume of Tally sync
2	Tally product, version, number of computers	Latest TallyPrime on a cloud Windows server in India, on 24 hours (hosting and licence to confirm)	Sync agent design
3	Fall-pico rules (limit before/after discount, online skip, worker type)	Per saree, after discount, customer may skip online, per-piece weekly pay	Order flow and wage calculation
4	Employee attendance method	Manager entry + PIN	Attendance screens

#	Decision	Current assumption	Build impact
5	WhatsApp provider and message budget	Official API provider, English/Hindi/Hinglish	Template approval takes days: start in Week 1
6	Shop hardware: counters, scanners, printers	One counter today (confirmed); scanner and printer models TBD	POS print and offline design
7	Tech stack	Decided: FastAPI, React + Vite + TypeScript, PostgreSQL, Celery + Redis	Team hiring and setup
8	Hosting	AWS Mumbai	Cost and data location

The full list is in SRS section 11.

2. Product Overview

2.1 Vision

FashionOS runs the client's whole fashion business from one system: billing at the shop, selling on the website, tracking one shared stock, handling customer credit (khata) with WhatsApp bills, paying staff and fall-pico workers, and keeping Tally books correct, with AI helping customers shop and the owner decide what to buy.

2.2 Goals

1. One stock for shop and website, so nothing is oversold and nobody counts twice.
2. Fast shop billing that keeps working when the internet is down.
3. Khata customers get an instant WhatsApp bill and can pay from home; the owner always knows who owes what.
4. Tally always matches FashionOS, without double entry by the accountant.
5. Staff salary, advances and fall-pico worker payments are clear and disputes are rare.
6. AI increases sales (recommendations, search, chat, posters) and improves buying decisions (trends, demand, missing products).

2.3 Users

User	Main needs
Owner	Full view of sales, cash, khata, stock, salary; approvals; buying decisions
Manager	Daily operations, discounts, attendance, fall-pico assignment
Cashier / sales staff	Quick billing, khata sales, returns
Accountant	Tally sync, reconciliation, GST, salary sheets
Online order staff	Pick, pack, ship website orders
Fall-pico supervisor and workers	Job list, quality check, earnings
Employee	Own attendance, salary, advance, balance
Khata customer	Own bills, balance, pay from home
Online customer	Browse, buy, track, return

2.4 Feature Map by Phase

Module	SRS requirements	Phase	What it delivers
Catalog, shared stock, barcodes	FR-CAT	1	One catalog and stock, labels, stock count
Shop billing (POS)	FR-POS	1	Billing, returns, cash register, offline mode
Khata, WhatsApp bills, pay from home	FR-KHT, FR-WAP, FR-PRT	1	Ledger, AI-worded bills, reminders, Pay Now
Tally integration	FR-TLY	1	Two-way sync, reconciliation, sync issues screen
Employees	FR-EMP	1	Attendance, salary, advance, statement
Fall-pico	FR-FPC	1	Jobs, tags, workers, QC, wages, cost
Purchasing (basic)	FR-PUR-01..03	1	Supplier bills, stock-in, labels
Website and online orders	FR-STO, FR-CHK, FR-PAY, FR-ORD	1 (basic)	Browse, cart, pay, ship, return
Access, audit, settings, reports	FR-AUT, FR-ADM, FR-RPT, FR-NOT	1	Roles, audit log, owner dashboard
AI: recommendations, chatbot, smart search, size, descriptions, poster	FR-AI-10..73	2	Customer and staff AI tools
AI: visual search, demand and trend, what to buy	FR-AI-50..52, 80..93	3	Buying intelligence

2.5 Success Metrics [*Assumption*] (targets to confirm)

Metric	Target
3-item cash bill at the counter	Under 30 seconds
Stock accuracy (system vs physical count)	98% or better
Tally vouchers synced within 5 minutes	99%, with zero duplicates and zero lost
WhatsApp bill delivered within 1 minute of khata sale	95%
Share of khata dues paid through Pay Now (after 3 months)	20%
Fall-pico jobs finished by promised date	90%
Website mobile load (4G)	Under 2.5 seconds
Online payment success rate	95%

3. Scope and Roadmap

3.1 In Scope (Phase 1)

- Shared catalog and stock, shop billing with offline mode, khata with WhatsApp bills and Pay Now.
- Tally integration in both directions, employees, fall-pico, basic purchasing.
- Basic website: browse, search, cart, checkout, payments, shipping, returns.
- Roles, audit log, owner dashboard and reports.

3.2 Out of Scope (for now)

Native apps, international sales, wholesale portal, AR try-on, Amazon/Flipkart sync, multiple shops, and a SaaS platform for other brands.

3.3 Roadmap [*Assumption*] (to be re-estimated)

Phase	Duration	Deliverables
Phase 0: Setup	Weeks 1 to 2	SRS approved, repos, CI, environments, Tally cloud server and version check, Tally data sample check, WhatsApp template request
Phase 1: Core	Weeks 3 to 18	Mode A first (Tally import and sync, khata, WhatsApp, Pay Now), then POS billing, employees, purchasing, fall-pico, website
POS parallel run and cutover	Weeks 13 to 20	Counter bills in FashionOS POS and in TallyPrime side by side, then cutover to POS (Mode B) after a clean run
Phase 2	8 to 10 weeks after go-live	AI recommendations, chatbot, smart search, size AI, description writer, poster maker, purchase orders
Phase 3	Later	Visual search, demand and trend forecast, what-to-buy, Hindi UI, more

3.4 Milestones

- M1 (Week 2):** SRS signed, designs and architecture approved, Tally cloud server access and version confirmed, Tally data sample checked.
- M2 (Week 6):** Mode A working on staging: Tally import (items, parties, vouchers), shared stock, Tally sync, Sync Issues screen.
- M3 (Week 10):** Khata, WhatsApp bills, Pay Now and Tally receipts live in the shop, while the counter still bills in TallyPrime.
- M4 (Week 16):** POS billing (offline), employees, purchasing and fall-pico working; POS parallel run in progress.
- M5 (Week 18):** Website checkout, orders and shipping working; UAT done.
- M6 (Week 20):** Parallel run clean, new bill series approved, POS cutover and go-live of everything.

3.5 Rollout Order

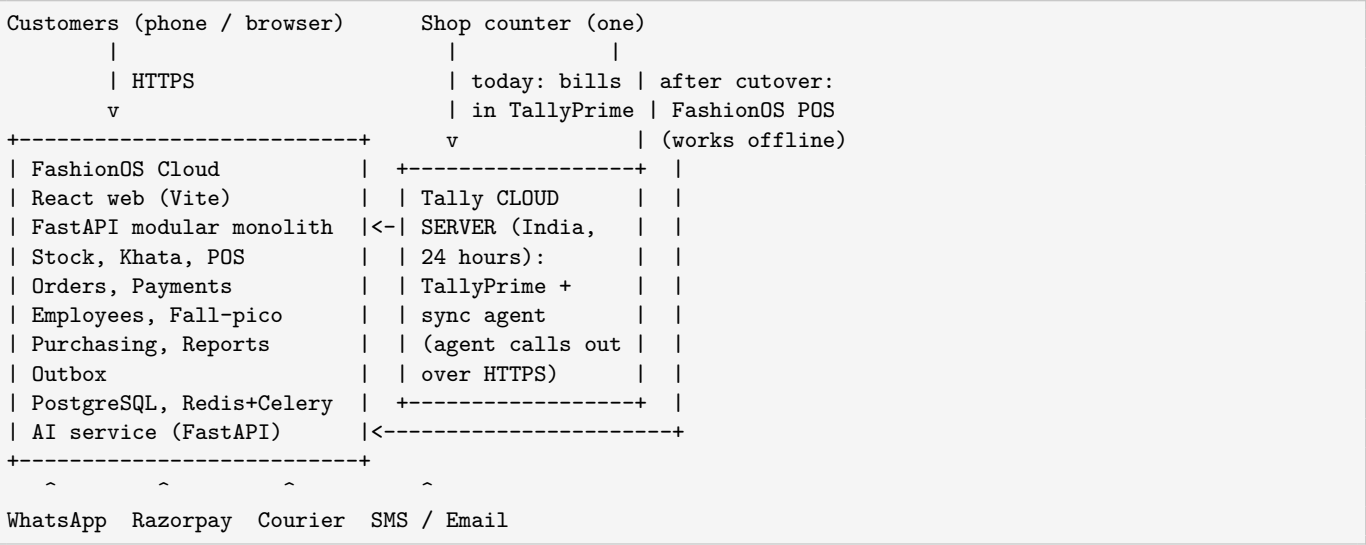
The shop already trades and bills in TallyPrime, so FashionOS goes live in steps and the counter's billing is never stopped:

- data import, shared stock and Tally sync while the counter keeps billing in TallyPrime (Mode A);
- khata, WhatsApp bills and Pay Now;
- POS billing, run side by side with Tally, then cutover (Mode B) after a clean parallel run;
- employees, fall-pico and purchasing;
- the website.

Each step runs in the shop for about a week before the next one starts.

4. System Architecture

4.1 High-Level Diagram



4.2 Components

Component	Role	Technology <i>[Assumption]</i>
Website and back office	Customer site, owner, manager, accountant, screens	React + Vite + TypeScript, Tailwind CSS (the public website needs prerendering or server rendering for SEO, TBD)
POS app	Shop billing, works offline	Installable web app (PWA) with local database (IndexedDB) and sync queue
API	All business rules	Python, FastAPI, SQLAlchemy 2.0, Alembic
Database	All records	PostgreSQL
Cache and queues	Sessions, jobs, retries	Redis, Celery
Tally sync agent	Talks to Tally on the cloud Tally server	Small Python service on the same server as Tally, using Tally's local XML/HTTP interface
WhatsApp gateway	Send and receive messages	Official WhatsApp Business API through a provider
AI service	WhatsApp wording, recommendations, chat, search, forecasts, posters	Python, FastAPI, hosted LLM and image model, pgvector
Files	Product and AI images	AWS S3 + CloudFront
Search	Product search	PostgreSQL full-text first; Meilisearch later
Monitoring	Errors, logs, alerts	Sentry, CloudWatch
CI/CD	Test and deploy	GitHub Actions, Docker

4.3 Key Design Decisions

- Modular monolith** for the backend. Simple for 5 people; modules can be split later.
- One stock ledger.** Every change to stock is a row in `stock_movements`; `stock_levels` is the current total. Shop, website, purchases, returns and Tally imports all write to it.
- Append-only money ledgers.** Khata, employee and worker balances are sums of entries. Entries are never edited or deleted; mistakes are fixed by reversing entries.

4. **Outbox pattern.** Anything that must reach Tally or WhatsApp is first saved in an outbox table in the same database transaction as the sale, then sent by a worker with retries. No sale is lost if Tally or WhatsApp is down.
5. **Offline-first POS.** The POS keeps its own copy of products, prices and customers, saves bills locally, and syncs when online. Each bill has a unique idempotency key, so a retry never creates a second bill.
6. **Tally agent is outbound only.** The agent runs on the same server as Tally and opens the connection to FashionOS. Tally's interface is never open to the internet.
7. **Tally is the master of books, FashionOS is the master of daily bills, stock and khata** (SRS 2.1). Conflicts go to a Sync Issues screen, never silently overwritten.
8. **AI is a separate service** with timeouts and fallbacks, so it can fail without stopping billing.
9. **Payments are confirmed only by gateway webhook**, never by the browser.
10. **Money is stored in paise** as whole numbers; GST is calculated on the server.
11. **Everything important is audited** (who, what, old, new, when, device).

4.4 Flow in Mode B (FashionOS POS): Khata Sale to WhatsApp and Tally

1. Cashier scans items, picks the khata customer, chooses khata payment.
2. POS checks credit limit (using the last synced balance) and warns if crossed.
3. POS saves the bill locally and sends it to the API with its idempotency key.
4. API in one transaction: saves bill, deducts stock, adds a khata ledger entry, creates fall-pico jobs for eligible sarees, writes outbox rows for Tally and WhatsApp.
5. Message worker asks the AI service for wording, checks every number against the ledger, picks an approved template and sends on WhatsApp (SMS if it fails).
6. Tally worker hands the sales voucher to the sync agent; the agent posts it to Tally and reports back; the outbox row is marked done.
7. Later the customer pays through the Pay Now link: Razorpay webhook, ledger entry, WhatsApp receipt, Tally receipt voucher.

4.5 Flow in Mode A (counter bills in TallyPrime; first step of rollout)

1. The cashier makes a khata bill in TallyPrime as today, with the customer's ledger as the party.
2. The sync agent on the Tally server reads the new voucher and sends it to the cloud (`/tally/inbox`).
3. The cloud matches the Tally party to a FashionOS customer (through `tally_ledger_map`), saves the sale (source `tally_import`), deducts stock and adds the khata ledger entry.
4. A WhatsApp job is created. Wording, validation and sending follow section 4.4. Delay: up to 5 minutes.
5. Saree lines above the fall-pico limit create fall-pico jobs.
6. Website stock is updated as soon as step 3 is done.
7. When the customer pays through the Pay Now link, FashionOS records the payment, sends the WhatsApp receipt, and the agent posts a Receipt voucher to Tally.
8. If the party or an item cannot be matched, the bill goes to the Sync Issues screen and no message is sent.

4.6 Environments

Environment	Purpose	Deploy
Local	Developer machine, Docker Compose, Tally demo company	Manual
Dev	Integrated testing	Auto on merge to develop
Staging	Client demo, UAT, test Tally company and WhatsApp sandbox	Auto on merge to release
Production	Live shop and website	Manual approval from main

5. Module Technical Notes

5.1 Shared Stock and Reservations (FR-CAT-02..04)

`stock_levels` holds `on_hand` and `reserved` per variant. A variant is sellable when `on_hand - reserved >= quantity`.

- Shop bill: deduct `on_hand`. Online order: add to `reserved` until paid, then deduct; release on cancel or after 30 minutes unpaid.
- Updates use one atomic statement or a row lock inside a transaction, so the last unit can only be sold once.
- **Offline POS case:** a counter without internet may sell a unit that was sold online in the meantime. The sync accepts the bill (it already happened), stock goes negative, and the owner gets a Stock Conflict alert. *[Assumption]* Accept and alert, never reject a finished shop sale.
- Stock count mode writes adjustment movements with a reason; nothing overwrites `on_hand` directly.

5.2 Offline POS (FR-POS-08)

- POS is an installable web app (PWA). A service worker caches the app; IndexedDB stores products, prices, barcodes, khata customers (with last synced balance and time), the bill queue and settings.
- Each counter has its own bill number series (for example C1-26-000123). The bill number plus counter ID is the idempotency key sent to the API.
- Sync runs every few seconds when online: send queued bills in order, then pull changes since the last sync token. Failed bills stay in the queue and are shown to the cashier.
- Credit limit check offline uses the cached balance and shows “balance may be outdated”. The owner can set a safer limit for offline mode.
- Printing: browser print for A4; a small local print helper or ESC/POS over USB/network for thermal printers *[TBD by printer model]*.
- Day-close needs all queued bills synced, or shows what is missing.

5.3 Khata Ledger (FR-KHT)

Table `ledger_entries` (append-only): `account_id`, `type` (SALE, PAYMENT, RETURN, ADJUSTMENT, WRITE_OFF, REVERSAL), `amount_paise` (positive = customer owes more), `ref_type`, `ref_id`, `note`, `created_by`, `running_balance_paise`.

- Writing an entry locks the account row, reads the last balance, writes the new balance. This keeps the running balance correct when two entries arrive together.
- Payments are allocated oldest bill first (`ledger_allocations`), so ageing is accurate.

- A reversal entry points to the entry it cancels; both stay visible.
- Daily integrity job: recompute every balance from entries and compare. Any difference raises an alert (NFR-DATA-01).
- Reminder rules (days after bill, due date, weekly) run as a scheduled job and respect quiet hours, opt-out and disputes.

5.4 WhatsApp Pipeline (FR-WAP)

1. Event (khata sale, payment, reminder, fall-pico ready) creates a `message_jobs` row with the facts needed (amounts, dates, bill number, balance).
2. The message builder calls the AI service with placeholders (for example `{amount}`, `{new_balance}`), the customer's language and tone, and asks only for wording.
3. The validator fills the placeholders from the database and checks that no other number appears in the text. If the check fails, the standard template is used.
4. The sender picks an approved template (or free text inside the 24-hour window) and calls the provider API.
5. Provider webhooks update status: sent, delivered, read, failed. A failure triggers SMS and flags the ledger entry.

Tables: `message_templates` (name, language, approval status), `whatsapp_optins`, `message_jobs`, `message_events`. Template names and wording are submitted to Meta in Phase 0; approval can take days.

5.5 Online Payments and Pay Now Links (FR-PAY, FR-PRT)

- Website checkout: create order (`PENDING_PAYMENT`), create Razorpay order, customer pays, webhook with verified signature marks it paid. Idempotency by gateway payment ID.
- Pay Now: a signed, unguessable token for one customer, valid 7 days, rate limited. It opens a page with balance and recent bills; paying creates a Razorpay payment for the chosen amount.
- On webhook: write a `PAYMENT` ledger entry, allocate to oldest bills, queue WhatsApp receipt and Tally receipt voucher. Duplicate webhooks do nothing.
- Overpayment becomes advance credit on the ledger.

5.6 Tally Integration (FR-TLY)

Parts: the outbox in the cloud, the sync agent on the Tally server, and mapping tables. TallyPrime runs on a cloud Windows server in India, on 24 hours (client decision; hosting and licence to confirm). The agent runs on that same server.

- **Two modes (SRS 3.18).** Mode A: the counter bills in TallyPrime and FashionOS imports the bills. Mode B: the counter bills in the FashionOS POS and FashionOS pushes sales to Tally. A `tally_mode` setting with a cutover date decides; only one mode is active, so a sale is never entered twice.
- **Tally to FashionOS** (Mode A, and always for purchases or stock entered in Tally): the agent reads new or changed vouchers and stock using Tally's change counter *[verify for the latest TallyPrime]* and sends the raw vouchers to `tally_inbox`. The cloud turns them into sales, returns, purchases and stock movements marked `tally_import`, and skips anything it created itself.
- **FashionOS to Tally:** the agent asks the cloud for pending outbox items over HTTPS, builds the Tally XML, posts it to Tally on the same server, then confirms success or error. Always sent: online orders, Pay Now receipts, shop receipts, supplier payments. In Mode B also shop sales and returns. Each voucher carries a unique FashionOS reference so a retry cannot create a second voucher.

- **Mapping tables:** `tally_item_map` (variant to stock item), `tally_ledger_map` (customer ledger name to phone; supplier, employee, GST, sales, cash, UPI and bank ledgers), `tally_voucher_types`.
- **Voucher types used:** Sales, Credit Note (returns), Receipt, Purchase, Debit Note, Payment, and Journal when needed.
- **Availability:** the server is on 24 hours. A setting holds Tally's working hours (default 24 hours; 11 am to 9 pm only if Tally stays on the shop computer). If Tally cannot be reached, items wait and retry with backoff; the alert goes out after 30 minutes inside working hours. A backlog clears within 15 minutes of Tally opening, and every voucher keeps its original date and time.
- **Errors** (missing ledger, locked period, unknown party or item) go to the Sync Issues screen with the reason. In Mode A a khata bill with an unmatched party sends no WhatsApp message.
- **Reconciliation:** a nightly job (and again after opening) compares day totals (sales, GST, receipts) and item stock between FashionOS and Tally and lists differences.
- **Bill numbers:** in Mode A numbers come from Tally. In Mode B FashionOS uses a new series from `bill_series` (prefix, financial year, counter, running number), unique and in order, designed with the accountant under GST rules. The old series stays closed in Tally.
- **Security:** the agent has its own API key and makes outbound HTTPS calls only. Tally's local interface is never open to the internet. The server is in India, open to named users (two-factor where the host offers it), with a daily backup of company data kept apart from the server.
- **Internet dependency:** while the counter bills in cloud Tally (Mode A), billing needs the internet. Keep a backup connection or the paper bill book (FR-TLY-27). This ends at cutover, because the POS works offline.
- **Go-live import:** items, ledgers and opening balances are read from Tally once and checked by the accountant, after a data quality check on a sample.

Verify in Sprint 0: the exact latest TallyPrime version on the server, whether the agent can run next to it, which requests are supported, and the licence for several users.

5.7 Employees and Payroll (FR-EMP)

```
Payable days = Present + Half-day x 0.5 + Paid leave + Weekly off (as per shop rule)
Monthly staff: Earned = Monthly salary x Payable days / Days in month
Daily staff:   Earned = Daily rate x Payable days
Gross          = Earned + Incentive + Overtime
Net payable    = Gross - Advance recovery - Other deductions
Pending        = Net payable - Salary already paid this month
```

Example: salary 15,000, 30-day month, 24 payable days, earned 12,000; advance recovery 1,000; net 11,000; already paid 6,000; pending 5,000. Advance of 5,000 falls to 4,000.

- Tables: `employees`, `attendance`, `salary_runs`, `salary_lines`, `advances`, `advance_recoveries`, `salary_payments`. All money tables are append-only with reversals.
- A salary run is a draft until the owner approves; after approval it is locked.
- Salary screens and APIs are allowed only for Owner, Accountant and the employee (own record).

5.8 Fall-Pico (FR-FPC)

- **Job states:** PENDING → ASSIGNED → IN_PROGRESS → COMPLETED → QC_PASSED → READY → HANDED_OVER. Side paths: REWORK (from QC fail), CANCELLED.
- On bill or paid order, for each saree line with `fall_pico_eligible = true` and price above the limit (setting, default 1,000), create one job and print the tag. The customer can decline at the counter; online customers can skip at checkout.

- **Assignment:** manual or lowest open jobs. Capacity per worker is a soft limit.
- **fp_job_events** records every handover and return (who, when, state), so a saree can always be found.
- **Wages:** a job becomes payable when it reaches QC_PASSED. Wage = worker rate at that time × number of pieces. Rework is not paid again. Worker statement: jobs done, earned, paid, advance, balance (same ledger design as employees).
- **Online orders:** after payment the order enters FALL_PICO_IN_PROGRESS; it can be packed only when the job is QC_PASSED or skipped.
- **Cost:** monthly cost = wages + material; report cost per saree.

6. AI Integration

6.1 AI Use Cases

ID	Feature	What it does	Phase	Model type
AI-0	WhatsApp bill and reminder wording	Writes polite message text around exact numbers	1	LLM (small, fast)
AI-1	Recommendations	Similar, also bought, complete the look, picked for you	2	Embeddings + behaviour data
AI-2	Support chatbot	Answers product, delivery, return, fall-pico and order questions	2	LLM with retrieval (RAG)
AI-3	Smart search	Natural language to filters plus meaning search	2	LLM parser + vector search
AI-4	Size recommendation	Suggests size from size chart, body details, past returns	2	Rules + ML
AI-5	Description writer	Title, description, tags from photos and details	2	Multimodal LLM
AI-6	Banner and poster maker	Designs from brief with exact editable text	2	Image model + template engine
AI-7	Visual search	Find similar products from a photo	3	Image embeddings
AI-8	Demand and trend forecast	7, 30, 90 day demand, trending items, reorder advice	3	Time-series + trend data
AI-9	What to buy	Missing products with evidence, quantity and budget	3	Analytics + LLM summary

6.2 AI Service Rules

- A separate service. The API calls it with a 3 second timeout (30 seconds for images) and always has a fallback.
- For anything about money, stock or order status, the answer comes from the database, not from the model.
- Personal data (full phone, address, salary, payment details) is never sent to an external model. Names are sent as first name or placeholder only.
- Each feature has an on/off switch, a monthly cost limit and usage logging.

6.3 Input and Output Examples

AI-0 WhatsApp bill wording (the model sees placeholders, not real amounts)

```
POST /ai/message/compose
{
  "purpose": "khata_bill",
  "language": "hinglish",
  "tone": "polite",
  "first_name": "Ramesh",
  "placeholders": ["{bill_no}", "{amount}", "{prev_balance}", "{new_balance}", "{pay_link}"]
}

{
  "text": "Namaste Ramesh ji, aapka bill {bill_no} ({amount}) ban gaya hai. Pehle ka baaki {prev_balance},
    ab kul baaki {new_balance}. Ghar se payment ke liye: {pay_link}. Dhanyavaad!",
  "source": "ai"
}
```

The validator fills the placeholders from the ledger and rejects the text if any other number appears or a placeholder is missing. Then the standard template is used.

AI-1 Recommendations

```
POST /ai/recommendations
{ "customer_id": "c_123", "context": "product_page", "product_id": "p_456", "limit": 8 }

{ "items": [ { "product_id": "p_789", "score": 0.91, "reason": "complete_the_look" } ], "source": "ai" }
```

AI-8 Demand forecast

```
{
  "product_id": "p_456",
  "horizon_days": 30,
  "forecast_units": 84,
  "confidence": "medium",
  "basis": "24 months of shop and online sales, wedding season uplift",
  "reorder_advice": { "suggested_units": 60, "reason": "stock lasts about 12 days" }
}
```

6.4 Models and Cost *[Assumption]*

Item	Choice	Note
Text and chat	Hosted LLM API	Small fast model for bill wording and chat; stronger model for descriptions and summaries
Image generation	Hosted image model	Text on posters added by the template engine, not drawn by the model
Embeddings and vector store	Hosted embeddings, pgvector	Move to a dedicated vector DB only if needed
Forecasting	Open-source time-series library	Trained on shop and online sales and imported Tally history
Cost control	Cache, token limits, per-feature monthly cap, alert at 80%	AI engineer to estimate cost per month
Vendor terms	No training on our data	Confirm before contract

6.5 Prompt Management

- Prompts live in `/ai/prompts/` as versioned files with purpose, variables, output format, owner and change log.
- A prompt change needs a pull request and a pass on the test set.

6.6 Fallbacks

Failure	What happens
AI wording fails or fails validation	Standard approved WhatsApp template is sent
Recommendations down	Bestsellers and recently viewed
Chatbot down	FAQ page, phone and WhatsApp contact
Smart search fails	Normal keyword search
Description or poster fails	Staff write or design manually; nothing is blocked
Forecast has too little data	Labelled low confidence; category-level trend only

6.7 Data Privacy for AI

- Send the minimum data. No salary, khata balances of named people, or payment details to external models (balances appear only as placeholders filled locally).
- Mask personal data in logs. Keep chat history 90 days; delete on request.
- Tell customers when they talk to an AI.
- Confirm the vendor's data policy before launch.

6.8 AI Quality Testing

- **WhatsApp wording:** 200 generated messages across languages and tones; zero wrong or missing amounts allowed.
- **Chatbot:** 100-question test set (FAQs, order status, other customers' orders, discount tricks, prompt attacks); 90% correct before launch.
- **Recommendations:** A/B test against bestsellers; measure click and add-to-cart.
- **Forecasts:** compare with real sales monthly and report error.

7. Database Schema

7.1 Main Tables

Group	Tables	Notes
People	staff_users, roles, customers, khata_accounts, addresses, whatsapp_optins	One customers row per person (unique phone); khata_accounts holds limit and credit days
Catalog and stock	products, variants, barcodes, categories, product_images, stock_levels, stock_movements, stock_counts	Variant has price, MRP, GST rate, HSN, cost, fall-pico flag
Shop sales	bills, bill_items, bill_payments, counters, cash_registers	bill_payments allows split payment (cash, UPI, card, khata)
Khata	ledger_entries, ledger_allocations, reminder_rules, disputes	Append-only
Online	carts, cart_items, orders, order_items, order_status_logs, shipments, returns, refunds, coupons, wishlists, reviews	Orders copy address and prices

Group	Tables	Notes
Payments	payments, payment_links, webhook_events	Unique gateway IDs prevent double processing
Purchasing	suppliers, purchases, purchase_items, purchase_orders	Purchase adds stock movements
Tally	tally_outbox, tally_inbox, tally_item_map, tally_ledger_map, sync_issues, reconciliation_runs	Outbox rows carry a unique voucher reference
WhatsApp and messages	message_templates, message_jobs, message_events	
Employees	employees, attendance, salary_runs, salary_lines, advances, advance_recoveries, salary_payments	
Fall-pico	fp_workers, fp_jobs, fp_job_events, fp_worker_ledger, fp_incidents, fp_materials	
AI	chat_sessions, chat_messages, product_embeddings, forecasts, buy_suggestions, ai_usage, missing_product_requests	
System	settings, audit_logs, feature_flags, files	

7.2 Relationships (simple view)

- A customer has many bills, orders and addresses, and at most one khata account, which has many ledger entries.
- A bill or order has many items; a saree item can create a fall-pico job, which has events, one worker and a wage entry.
- A variant has one stock level and many stock movements from bills, orders, purchases, returns and Tally imports.
- An employee has attendance rows, salary lines, advances and payments.
- Every bill, payment, return, purchase, salary and wage can have outbox rows for Tally.

7.3 Database Rules

- UUID primary keys; `created_at`, `updated_at`, `created_by` on every table; `tenant_id` is not needed (single business).
- Money in paise (integers). Dates in UTC, shown in IST.
- Financial and stock tables are append-only: no update or delete; reversals only.
- Products and customers use soft delete.
- Unique constraints on bill number per counter, voucher reference, gateway payment ID and idempotency keys.
- Indexes on `bills(created_at)`, `ledger_entries(account_id, created_at)`, `stock_movements(variant_id, created_at)`, `tally_outbox(status, created_at)`, `fp_jobs(status, worker_id)`.
- Migrations through a tool; no manual edits in staging or production. Row-level access checks for salary and khata data.

Added in v1.0: `bill_series` (prefix, financial year, counter, next number) holds the new bill numbering. Every bill and stock movement has a `source` (`pos`, `website`, `tally_import`) so Mode A and Mode B can be told apart. `tally_inbox` stores the raw vouchers read from Tally. A `tally_mode` setting (A or B) records the mode and the cutover date.

8. API Documentation

Base URL: `/api/v1` **Format:** JSON **Auth:** Authorization: Bearer <token> for staff, customers and workers; API key for the Tally agent; signed token for Pay Now links.

8.1 Standard Response Format

```
{ "success": true, "data": { }, "error": null }

{ "success": false, "data": null,
  "error": { "code": "CREDIT_LIMIT_EXCEEDED",
            "message": "Limit crossed by 1200. Manager approval needed." } }
```

8.2 Common Error Codes

HTTP	Code	Meaning
400	VALIDATION_ERROR	Wrong or missing input
401	UNAUTHORIZED	Not logged in or token expired
403	FORBIDDEN	Role not allowed
404	NOT_FOUND	Item does not exist
409	OUT_OF_STOCK / DUPLICATE	Conflict; same idempotency key returns the original result
422	CREDIT_LIMIT_EXCEEDED / COUPON_INVALID	Business rule failed
429	RATE_LIMITED	Too many requests
500	SERVER_ERROR	Unexpected error (sent to Sentry)

8.3 Endpoints

Module	Method	Endpoint	Who	Purpose
Auth	POST	<code>/auth/login</code> , <code>/auth/otp/send</code> , <code>/auth/otp/verify</code> , <code>/auth/refresh</code>	All	Login
Catalog	GET/POST/ PATCH	<code>/products</code> , <code>/products/:id</code> , <code>/variants/:id</code>	Staff / public read	Products
Catalog	POST	<code>/products/import</code>	Staff	CSV import
Stock	GET	<code>/stock?variant=</code>	Staff	Current stock
Stock	POST	<code>/stock/adjustments</code> , <code>/stock/counts</code>	Manager	Adjust, count
POS	GET	<code>/pos/sync?since=token</code>	Counter	Pull products, prices, customers
POS	POST	<code>/pos/bills</code>	Counter	Push bill (idempotent)
POS	POST	<code>/pos/bills/:id/return</code>	Cashier + approval	Return or exchange

Module	Method	Endpoint	Who	Purpose
POS	POST	/pos/registers/open, /pos/registers/close	Cashier	Cash register
Customers	GET/POST/ PATCH	/customers	Staff	Find and create
Khata	POST	/khata/accounts	Manager	Open khata
Khata	GET	/khata/accounts/:id/ledger	Staff	Ledger
Khata	POST	/khata/accounts/:id/payments	Cashier	Record payment in shop
Khata	POST	/khata/accounts/:id/adjustments	Owner	Reversal, write-off
Khata	GET	/khata/reports/ageing	Owner	Ageing
Portal	GET	/pay/:token	Customer	Balance and bills for Pay Now
Portal	POST	/pay/:token/start	Customer	Start a payment
Payments	POST	/webhooks/razorpay	Signature	Payment confirmation
Messages	POST	/webhooks/whatsapp	Signature	Delivery status, replies
Messages	POST	/messages/:id/resend	Staff	Resend bill
Website	GET	/store/products, /store/search, /store/categories	Public	Browse
Website	CRUD	/store/cart, /store/checkout, /store/orders	Customer	Buy
Orders	GET/PATCH	/orders, /orders/:id/status	Staff	Online order handling
Shipping	POST	/orders/:id/shipment	Staff	Create courier shipment
Purchasing	POST/GET	/purchases, /suppliers	Staff	Stock-in
Employees	CRUD	/employees, /employees/:id/attendance	Manager/ Owner	Staff records
Payroll	POST	/payroll/runs, /payroll/runs/:id/approve	Owner/ Accountant	Salary
Payroll	POST	/employees/:id/advances, /employees/:id/salary-payments	Owner	Advance, payment
Payroll	GET	/me/statement	Employee	Own statement
Fall-pico	GET/POST	/fp/jobs, /fp/jobs/:id/assign, /fp/jobs/:id/status	Supervisor	Job flow
Fall-pico	GET/POST	/fp/workers, /fp/workers/:id/payments	Supervisor/ Owner	Workers and wages
Tally agent	GET	/tally/outbox/pending	Agent key	Pull work
Tally agent	POST	/tally/outbox/:id/result	Agent key	Report success or error
Tally agent	POST	/tally/inbox	Agent key	Send changes made in Tally
Tally	GET	/tally/issues, /tally/reconciliation	Accountant	Sync issues and reports
AI	POST	/ai/* (internal)	API only	AI service
Reports	GET	/reports/dashboard, /reports/sales, /reports/gst	Owner/ Accountant	Reports
Settings	GET/PATCH	/settings	Admin/Owner	Rules and integrations

8.4 Sample: Push a Khata Bill from POS

```
POST /api/v1/pos/bills
{
  "idempotency_key": "C1-26-000123",
  "counter_id": "C1",
  "customer_id": "c_123",
  "items": [ { "variant_id": "v_88", "qty": 1, "price": 249900 } ],
  "payments": [ { "mode": "CASH", "amount": 100000 },
    { "mode": "KHATA", "amount": 149900 } ],
  "sold_by": "staff_7",
```

```

    "created_at_device": "2026-10-05T11:42:10+05:30"
  }
}

Response:
{
  "success": true,
  "data": {
    "bill_id": "b_9001",
    "bill_no": "C1-26-000123",
    "khata_new_balance": 349900,
    "fall_pico_jobs": [ "fp_2210" ],
    "queued": [ "whatsapp", "tally" ]
  },
  "error": null
}

```

Sending the same `idempotency_key` again returns this same response and creates nothing new. Document the full API in Swagger/OpenAPI (generated by FastAPI at `/docs`) and keep a Postman collection in the repo.

9. UI/UX Guidelines

9.1 Interfaces

Interface	Users	Notes
Website	Online customers	Mobile first, fast, SEO friendly
Back office	Owner, manager, accountant, online order staff, supervisor	Desktop first, works on tablet
POS	Cashiers	Full screen, scan first, big buttons, works offline
Simple pages	Workers, employees, khata customers	Phone friendly, very few options, large text, Hindi labels where needed

9.2 Sitemap

Website

- Home → Category → Product → Cart → Checkout → Confirmation
- Search, Account (profile, addresses, orders, wishlist), Login
- Help and policies (returns, shipping, fall-pico, privacy, terms, contact), AI chat

POS

- Billing → Customer → Payment (cash / UPI / card / khata / split) → Print or WhatsApp
- Hold bills, Returns, Cash register, Day close, Sync status

Back office

- Dashboard
- Catalog (products, stock, labels, stock count)
- Sales (bills, online orders, returns, shipments)
- Khata (customers, ledgers, ageing, reminders, messages)
- Purchasing (suppliers, purchases)
- People (employees, attendance, salary, advances)

- Fall-pico (job board, workers, wages, cost)
- Tally (sync status, issues, reconciliation, mapping)
- Reports, AI tools (poster maker, descriptions, trends, what to buy), Settings

Simple pages

- Customer: Pay Now → balance → pay → receipt; ledger and bills
- Employee: my attendance, salary, advance, balance
- Worker: my jobs, my earnings

9.3 Design System [*Assumption*] (designer to finalise)

Item	Rule
Style	Website: clean, fashion-first, large product photos. Back office and POS: plain, fast, high contrast
Colors	Primary black #111111; accent to be chosen from brand; success #1E8E3E; warning #F9AB00; error #D93025; background #FFFFFF / #F7F7F7
Fonts	Website heading: modern serif or display font; body and tools: Inter or similar, with Devanagari fallback
Sizes	12, 14, 16, 20, 24, 32 px; POS text at least 16 px
Spacing	4 px grid
Buttons	Primary, secondary, text; minimum height 44 px; POS main buttons at least 56 px
Money display	Always in INR with thousands separators (for example 1,49,900 paise is shown as 1,499), negative balances clearly marked
Status colors	Same colors everywhere: green done, amber pending, red overdue or error
Responsive	Mobile first; breakpoints 360, 768, 1024, 1440 px
Accessibility	Contrast 4.5:1, alt text, keyboard use (POS shortcuts)
States	Every component needs hover, focus, disabled, loading, empty, error and offline states

9.4 Key User Flows

1. **Cash bill:** Scan items → discount if allowed → cash → print or WhatsApp → next bill. No more than 3 screens.
2. **Khata bill:** Find customer by phone → scan items → choose khata → see balance and limit → confirm → bill sent on WhatsApp.
3. **Customer pays dues:** Open WhatsApp link → see balance → choose amount → pay UPI → receipt.
4. **Saree with fall-pico:** Bill saree → prompt: fall-pico yes → tag printed → supervisor assigns → worker finishes → QC → customer told.
5. **Month-end salary:** Check attendance → review salary draft → see advance recovery → approve → record payment.
6. **Online order:** Search or browse → product → size help → cart → checkout → pay → track.

9.5 Design Deliverables

Figma file with design system, POS screens (counter and tablet), back-office screens, website (desktop and mobile), simple pages, offline and error states, and a clickable prototype for flows 1 to 4. Developers get

Dev Mode access. The POS flow is tested with a real cashier before development continues.

10. Team Roles and Sprint Plan

10.1 Roles [*Assumption*] (5 members)

Role	Owns	Also helps with
Frontend and POS developer	POS app (offline), back-office UI, website, simple pages	Design QA
Backend developer (lead)	Core API, stock, billing, khata ledger, payroll, fall-pico, database, hosting	CI/CD
Integration developer	Tally sync agent, WhatsApp, Razorpay, courier, SMS and email	Reconciliation and reports
AI engineer	AI service, WhatsApp wording and validator, recommendations, chatbot, forecasts	Data pipelines, search
PM, QA and design lead	SRS, planning, Figma, test cases, UAT, client and accountant communication	Training and handover

One person doing PM, QA and design is a lot. A part-time designer or freelancer for Figma work is recommended [*TBD*].

10.2 Sprint Plan (2-week sprints) [*Assumption*] (to be re-estimated after SRS sign-off)

S	Weeks	Focus	Frontend / POS	Backend	Integration	AI	PM / QA / Design
0	1-2	Setup	Repo (pnpm), UI base, route shells	FastAPI base, database, auth, CI	Check latest TallyPrime on the cloud server, test agent, WhatsApp template request, Razorpay sandbox	AI service skeleton, vendor keys	SRS sign-off, Figma system, Tally data sample check, test plan
1	3-4	Import and stock	Product and stock screens, labels	Catalog, stock ledger, CSV import, bill series settings	Agent reads Tally: items, parties (khata customers), opening balances	Event tables	Data check report, stock test cases
2	5-6	Tally sync (Mode A)	Sync status, Sync Issues and reconciliation screens	Import sales, returns, purchases and stock from Tally; stock movements	Tally to FashionOS import; online orders and receipts to Tally	Reports queries	Mode A tests, M2
3	7-8	Khata and WhatsApp	Khata and ledger screens	Ledger from imported bills, limits, reminders, ageing	WhatsApp send and webhooks; match Tally party ledgers to customers	Bill wording and validator	Ledger tests
4	9-10	Pay from home	Pay Now pages	Payment links, allocation, receipts	Razorpay links and webhooks, Tally receipt vouchers	200-message wording test	Payment tests, M3 (khata and WhatsApp live in the shop)
5	11-12	POS billing	POS billing screen, offline queue, printing	Bills, GST, bill series, returns, cash register	Tally sales vouchers from POS (Mode B), scanner and printer	Data pipelines	Offline test plan
6	13-14	Employees, purchasing, POS parallel run	Attendance, salary, purchase screens	Payroll engine, advances, purchases	Tally purchase and salary vouchers	Data pipelines	Payroll tests with accountant; parallel run starts

S	Weeks	Focus	Frontend / POS	Backend	Integration	AI	PM / QA / Design
7	15-16	Fall-pico	Job board, worker pages, tags	Jobs from POS and imported bills, wages, QC	Courier setup, Tally wage entries	Recommendation data pipeline	Fall-pico tests, M4
8	17-18	Website	Storefront, cart, checkout, account, tracking	Store APIs, orders, returns, fall-pico online stage	Razorpay checkout, Shiprocket, settlement match	Search indexing, fallback recommendations	UAT, M5
9	19-20	Hardening and cutover	Fixes, performance, SEO (prerender)	Security, load test, cutover rehearsal	Reconciliation checks, Tally backup checks	AI tuning	Training, bill series approval, cutover, M6

Phase 2 (AI features) starts after go-live.

10.3 Task Board Rules

- Tool: Jira, Trello or Notion *[TBD]*. Columns: Backlog, Ready, In Progress, In Review, QA, Done.
- Every task has an owner, estimate, due date, linked requirement ID (for example FR-KHT-02) and acceptance criteria from the SRS.
- **Definition of Done:** code reviewed, tests pass, deployed to dev, QA checked, SRS and this document updated.
- Daily 10-minute standup; sprint planning and review every 2 weeks; demo to the client and the accountant at each milestone.

11. Developer Guide

11.1 Repository Structure

```

fashionos/
  apps/
    web/                # React + Vite + TypeScript
      src/routes/       # storefront | backoffice | pos | portal
      src/features/    # catalog, inventory, khata, pos, fall-pico, tally, ...
      src/lib/         # api, auth, permissions, i18n, offline, money
    api/               # FastAPI modular monolith
      app/core/        # config, db, money, audit, outbox, idempotency
      app/modules/     # auth, catalog, inventory, customers, pos, khata, payments,
                      # orders, shipping, purchasing, employees, fall_pico,
                      # notifications, tally, analytics, settings
                      # (each: router, schemas, service, repository, models, events)
      app/integrations/ # razorpay, whatsapp, shiprocket, sms, email, ai_client
      alembic/         # database migrations
      tests/
    ai-service/        # FastAPI: recommend, demand, embeddings, banners, vision, evals
    worker/           # Celery: outbox, reminders, tally_sync, reconciliation
    tally-bridge/      # Python agent on the Tally cloud server
  packages/          # shared-types, config
  database/          # seeds, policies (append-only SQL)
  docs/              # SRS, ADRs, runbooks
  infrastructure/    # docker, caddy, deployment, monitoring
  tests/             # e2e, contracts, load, offline-sync, tally-sync
  scripts/           # new_module.py
  docker-compose.yml, .env.example, pnpm-workspace.yaml

```

This is the layout of the project skeleton (`FashionOS_project_skeleton.zip`). Add files in the build order; do not create empty files just to match the tree.

11.2 Coding Standards

- Backend and AI service: Python 3.11+ with type hints everywhere and Pydantic for all input and output. Web: TypeScript (strict). Tally agent: Python.
- Format and lint: Ruff and Black (Python); ESLint and Prettier (web). Pre-commit hooks and CI enforce them.
- Names: `snake_case` in Python and database columns; `camelCase` variables and `PascalCase` components in TypeScript; `kebab-case` folders and files in the web app.
- Money is always an integer in paise, never a float (`app/core/money.py`, `src/lib/money.ts`).
- Every API input is validated. No secrets or personal data in logs.
- Anything that writes to stock, ledgers, salary or the outbox goes through a service function with one transaction and an audit entry. Routers never write to tables directly.
- Every bill and every webhook uses an idempotency key.
- Ledger, stock movement and audit tables are append-only, enforced by `database/policies`. Fix mistakes with a reversing entry.
- Database changes only through Alembic migrations.
- Comment the *why*, not the *what*.

11.3 Git Workflow

Branch	Purpose
<code>main</code>	Production, protected
<code>release</code>	Staging
<code>develop</code>	Integration
<code>feature/FR-KHT-02-credit-sale</code>	New work, from <code>develop</code>
<code>fix/webhook-duplicate</code>	Bug fixes

- Commits: `feat(khata): add credit limit check` (Conventional Commits).
- A pull request needs 1 approval, and 2 approvals for money, stock, ledger, payroll and Tally code, plus passing CI.
- Keep pull requests under 400 changed lines. Squash merge.

11.4 Environment Setup (new developer, target 30 minutes)

1. Install Python 3.11+, Node 20+, pnpm and Docker.
2. Clone the repo and copy `.env.example` to `.env`; get sandbox keys from the team lead.
3. Run `docker compose up -d` (PostgreSQL with pgvector, Redis).
4. API: in `apps/api` create a virtual environment, run `pip install -r requirements.txt`, `alembic upgrade head`, then `uvicorn app.main:app -reload -port 8000`. API docs open at `http://localhost:8000/docs`.
5. Web: from the repo root run `pnpm install`, then `pnpm dev:web` (opens on `http://localhost:5173`).
6. Worker and AI: in `apps/worker` run `celery -A app.celery_app worker -B`; in `apps/ai-service` run `uvicorn app.main:app -port 8001`.
7. Tally work: use a TallyPrime demo company on a Windows machine or a test cloud server and run the agent in `apps/tally-bridge` against it. **Never test on the real company data.**
8. Run the API tests with `pnpm test:api`.

11.5 Environment Variables

Variable	Purpose
DATABASE_URL, REDIS_URL	Database and queue
JWT_SECRET, JWT_REFRESH_SECRET, PAYLINK_SECRET	Tokens (login, Pay Now links)
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET	Payments
WHATSAPP_PROVIDER_URL, WHATSAPP_API_KEY, WHATSAPP_WEBHOOK_SECRET	WhatsApp
SMS_API_KEY, SMS_SENDER_ID, SES_*	SMS and email
SHIPROCKET_EMAIL, SHIPROCKET_PASSWORD	Courier
TALLY_AGENT_API_KEY	Agent identity (set on the agent, checked by the API)
AWS_S3_BUCKET, AWS_ACCESS_KEY, AWS_SECRET_KEY	Files
AI_API_KEY, AI_MODEL_TEXT, AI_MODEL_EMBED, AI_MODEL_IMAGE, AI_SERVICE_URL, AI_MONTHLY_BUDGET	AI service
SENTRY_DSN	Error tracking

Never commit `.env`. Keep keys in a secrets manager for staging and production.

12. Security and Performance Checklist

12.1 Security

#	Check	Owner	Done
1	HTTPS everywhere, HSTS	DevOps	☐
2	Passwords hashed (argon2 or bcrypt); 2FA for Owner and Admin	Backend	☐
3	Role checks on every API and screen; employee, worker and customer see only their own data	Backend	☐
4	Salary and khata data encrypted at rest; access logged	Backend	☐
5	Append-only ledgers: database permissions block update and delete on ledger and stock tables	Backend	☐
6	Audit log complete and not editable	Backend	☐
7	Pay Now tokens unguessable, expiring, rate limited, show only that customer's data	Backend	☐
8	Razorpay and WhatsApp webhooks verified by signature; duplicates ignored	Integration	☐
9	Tally agent: own API key, outbound HTTPS only, key revocable, signed updates	Integration	☐
10	Prices, discounts and GST calculated on server only	Backend	☐
11	POS local data (IndexedDB) holds no card data; devices lock after inactivity; owner can sign out a lost device	Frontend	☐

#	Check	Owner	Done
12	WhatsApp opt-in recorded; opt-out honoured; quiet hours respected	Integration	☐
13	Input validation and sanitising (injection, XSS), CSRF protection	All	☐
14	Rate limits on login, OTP, payment links, chatbot	Backend	☐
15	Secrets in a secrets manager; keys rotated before launch	DevOps	☐
16	File upload checks (type, size, scan)	Backend	☐
17	AI tests: prompt injection, data leak, wrong amounts	AI	☐
18	Dependency scans in CI	DevOps	☐
19	Daily backups; restore tested; backup of Tally data confirmed with the accountant	DevOps	☐
20	Privacy notice, consent, deletion process for customers, employees and workers	PM	☐

Added for Tally on the cloud (v1.0):

#	Check	Owner	Done
21	Cloud Tally server in India, open only to named users, two-factor where the host offers it	DevOps + Accountant	☐
22	Tally's local interface not reachable from the public internet; only the agent uses it	Integration	☐
23	Daily Tally company backup kept apart from the server; restore tested	DevOps	☐
24	Agent set to start automatically; alert if it stops	Integration	☐
25	Remote access to the Tally server reviewed: no shared logins, record of who logged in	Accountant	☐

12.2 Performance

Target	Value
POS: item appears after scan	Under 1 second
Website LCP (mobile, 4G)	Under 2.5 seconds
API response (95th percentile)	Under 400 ms
Load test	1 counter billing (tested up to 3) plus 500 shoppers and 100 orders per minute, errors under 1%
Sync	POS queue clears within 1 minute of internet returning

Techniques: CDN, WebP images, Redis cache for product lists, database indexes, server rendering for website pages, local cache in POS.

12.3 SEO Checklist (website)

- Unique title and description per page; clean URLs (/sarees/silk/kanjivaram-red).
- Product structured data (price, availability, reviews); `sitemap.xml`, `robots.txt`, canonical and Open Graph tags.
- Image alt text; fast mobile pages.

13. Test Plan

13.1 Test Types and Tools

Type	What	Tool <i>[Assumption]</i>
Unit	Ledger, stock, payroll, GST, fall-pico rules	Jest, Pytest
API	Endpoints, errors, idempotency	Postman/Newman, Supertest
End-to-end	POS, khata, website journeys	Playwright
Offline	POS with internet cut and restored	Playwright with network off, real counter test
Integration	Tally (demo company), WhatsApp sandbox, Razorpay test mode	Scripted tests + manual checks by accountant
Reconciliation	FashionOS vs Tally totals	Daily report during staging
Performance	Load	k6
Security	Vulnerabilities, access	OWASP ZAP, manual review
AI quality	Wording, chatbot, recommendations	Test sets + review
UAT	Owner, cashier, accountant, supervisor	Staging

13.2 Core Test Cases (examples; full list tracked in the test tool)

ID	Area	Test	Expected result
T01	POS	3-item cash bill	Bill printed, stock down on website, Tally sales voucher within 5 min
T02	POS	Split payment: cash + UPI + khata	Each part recorded; khata part added to ledger
T03	POS	Discount above role limit	Manager PIN asked; approval logged
T04	POS	Internet off, make 5 bills, turn on	All 5 synced once, no duplicate numbers
T05	Stock	Last unit bought at shop and online together	One succeeds; other gets clear message
T06	Khata	Credit sale within limit	Ledger entry; WhatsApp bill with correct amounts in 1 minute
T07	Khata	Credit sale over limit	Blocked; manager override logged with reason
T08	WhatsApp	AI text has a wrong amount	Standard template sent with correct amount
T09	WhatsApp	Customer without opt-in, or number not on WhatsApp	No message / SMS fallback and flag
T10	Pay Now	Customer pays part amount	Balance falls, receipt on WhatsApp, Tally receipt voucher
T11	Pay Now	Same webhook delivered twice	Only one ledger entry
T12	Pay Now	Expired link or another customer's link	Access denied
T13	Ledger	Try to edit or delete an entry	Denied; reversal works and both show
T14	Ledger	Daily integrity check on a deliberately changed test balance	Difference reported
T15	Tally	Tally closed, 20 bills made, Tally opened	20 vouchers synced once each, in order
T16	Tally	Sale entered directly in Tally	Website and shop stock drop within 5 minutes
T17	Tally	Deliberate mismatch in staging	Reconciliation report lists it
T18	Returns	Return of a khata bill	Balance reduced; credit note in Tally
T19	Payroll	24 payable days, salary 15,000, advance 5,000 with 1,000 recovery, 6,000 paid	Earned 12,000; net 11,000; pending 5,000; advance balance 4,000

ID	Area	Test	Expected result
T20	Payroll	Advance above limit	Owner approval needed
T21	Privacy	Employee views own statement; manager tries salary	Own only; manager denied
T22	Fall-pico	Saree at 1,000 and at 1,001	No job at 1,000; job at 1,001
T23	Fall-pico	Job with QC fail and rework	Rework not paid twice; worker statement correct
T24	Fall-pico	Online saree order	Cannot be packed until QC passed or skipped
T25	Website	UPI order success	Webhook confirms; GST invoice; Tally voucher
T26	Website	Payment fails then retry; double click on Pay	One order, one charge
T27	Website	COD order and remittance	Matches courier remittance report
T28	Website	Online customer at checkout	No khata or credit option
T29	Access	Cashier tries a write-off	Denied (403)
T30	Mobile	Full purchase on a 360 px phone	No layout break
T31	Performance	Load test from 11.2	Errors under 1%
T32	AI	AI service off	Billing, checkout, payments work; fallbacks shown
T33	AI (Phase 2)	Chatbot asked for another person's order or a 90% discount	Refused politely

Added tests for Mode A and cloud Tally (v1.0):

ID	Area	Test	Expected result
T34	Mode A	Sale made in TallyPrime	Website stock drops within 5 minutes; sale saved once
T35	Mode A	Khata bill made in TallyPrime	WhatsApp bill after import, amounts match the Tally ledger
T36	Mode A	Khata bill with an unmatched party	Goes to Sync Issues; no message sent
T37	Bill series	Bills made in Mode B	New series is unique and in running order; old series untouched
T38	Cloud Tally	Tally server restarted	Agent starts by itself; waiting items clear in order, no duplicates
T39	Cloud Tally	Tally unreachable for over 30 minutes	Alert is sent; items wait; nothing lost
T40	Counter	Internet off while billing in cloud Tally	Paper bill book used; bills entered later with original date and time
T41	Mode A	Saree above the fall-pico limit billed in Tally	Fall-pico job created from the imported bill

13.3 Bug Severity

Level	Meaning	Fix time
Critical	Wrong money, stock or ledger; lost or duplicate bills or vouchers; data leak; billing down	Same day
High	Main feature fails, no workaround	2 days
Medium	Feature fails, workaround exists	Current sprint
Low	Cosmetic	Backlog

A bug report has steps, expected vs actual result, screenshot or video, device or browser, and severity.

13.4 Release Gate (must be true to go live)

- No Critical or High bugs open; T01 to T41 pass on staging.

- The two-week parallel run shows zero unexplained difference between FashionOS and Tally totals and between FashionOS and the shop's old khata book.
- Security checklist done; backup restore tested.
- Message templates approved; payment gateway live; courier account live.
- Owner, accountant and shop staff sign off UAT.

14. Deployment and Go-Live Guide

14.1 Release Steps (Production)

1. Freeze code on **release**; confirm staging passed the release gate (12.4).
2. Merge **release** into **main** and tag the version (for example **v1.0.0**).
3. Take a database backup.
4. CI builds Docker images and runs tests.
5. Tech lead and PM approve the deploy.
6. Run database migrations (backward compatible only).
7. Deploy API, AI service, web and POS (rolling, no downtime). POS devices pick up the new version at next start; a version check warns if a device is too old.
8. Smoke test: log in, make a cash bill, make a khata bill (check WhatsApp and Tally), pay through Pay Now with a small amount, place a test website order, check dashboard.
9. Watch Sentry and logs for 1 hour; tell the team and the client.

14.2 Installing the Tally Sync Agent (Cloud Tally Server)

1. Confirm the host of the Tally cloud server (India), the latest TallyPrime version, the licence for several users (counter, accountant, agent), and who may log in.
2. Back up the Tally company data.
3. Move the company data to the server and check it with the accountant (counts of ledgers and stock items, closing balances).
4. Install the agent as a Windows service on the same server. Set it to start with the server and to keep running when users log out. Enter the API key from FashionOS settings.
5. Keep Tally open with the company loaded and its local XML/HTTP access enabled [*verify for the latest TallyPrime*]. This interface is only for the agent; never open it to the internet.
6. Run the mapping setup (ledgers, GST ledgers, voucher types, godown) and the check screen.
7. Test on a test company first: send a test sales voucher and a test receipt, and read some vouchers back (Tally to FashionOS). The accountant checks them.
8. Switch to the real company only after the accountant approves, and start in Mode A.
9. Write the runbook: how to restart the agent, where its log is, what to do when Tally is closed or the server restarts, and how to restore from backup.

14.3 Data Migration and Parallel Run

Data	Source	Check
Items and stock	Tally stock items and a physical count	Count matches system on a sample of 100 items
Customers and khata balances	Tally party ledgers (Sundry Debtors); khata is kept in Tally	Every balance signed off by the owner; total matches Tally Sundry Debtors
Suppliers	Tally	Totals match
Employees, advances, salary history	Owner's records	Owner and each employee confirm their balance
Fall-pico workers and open jobs	Supervisor's register	Open jobs entered with tags

Parallel run (2 weeks): the shop bills in FashionOS and also keeps the old method (Tally entry or paper bill). Every day the accountant compares the day totals, khata changes and stock. Any difference is explained and fixed before go-live. Customer WhatsApp messages stay on for a few friendly customers first, then everyone.

14.4 Rollback and Contingency

- **Software problem after a release:** redeploy the previous image tag; if a migration caused it, run the down-migration or restore the backup; switch off new features with feature flags (AI can be switched off without redeploy). Decision owner: tech lead; PM and client told within 15 minutes.
- **Internet problem at the counter while billing in cloud Tally (Mode A):** use the backup internet connection, or the paper bill book and enter the bills later with their original date and time (FR-MIG-08).
- **After cutover (Mode B):** the POS keeps billing offline. If the POS itself fails, use the paper bill book, or bill in Tally and let FashionOS import the bills (FR-TLY-07).
- **Tally server or agent problem:** items wait in the outbox and nothing is lost. Fix and restart the agent, restore Tally from backup if needed, then check the Sync Issues screen.
- **WhatsApp provider outage:** SMS fallback; reminders pause automatically.
- **Cutover rollback:** switch the counter back to Mode A (billing in Tally) within one working day, and export the bills made in FashionOS to Tally.
- Write a short incident note after every incident: what happened, impact, fix, how to prevent.

14.5 Monitoring and Alerts

- Uptime check every minute on website and `/health`; alerts to Slack or WhatsApp for 5xx spikes, payment failures, slow responses, database or Redis down, POS sync backlog, Tally outbox older than 30 minutes, WhatsApp failures above 5%, AI cost above 80% of budget.
- Dashboards: bills per hour, sync backlog, Tally sync success, message delivery, payment success, AI latency and cost.

14.6 Go-Live Checklist

Item	Owner	Done
Domain, SSL, DNS ready	DevOps	☐
Razorpay live, webhook URL set	Integration	☐

Item	Owner	Done
WhatsApp templates approved; opt-ins recorded for khata customers	Integration + Owner	☐
SMS DLT registration done	Integration	☐
Courier account live	Integration	☐
Tally agent running; mapping approved; test vouchers matched	Integration + Accountant	☐
Opening stock, customers, balances, staff data signed off	Owner + Accountant	☐
Parallel run shows no unexplained difference	PM + Accountant	☐
Barcode labels on stock; scanners and printers tested at each counter	PM	☐
Staff trained (14.6); paper bill book ready as backup	PM	☐
Admin accounts created; default passwords removed; 2FA on for Owner	Backend	☐
Backups and monitoring active; restore tested	DevOps	☐
Policy pages, privacy notice and return rules published	PM + Owner	☐

Added for the Tally cloud move and the new bill series (v1.0):

Item	Owner	Done
Tally cloud server live in India; latest TallyPrime; licence for several users confirmed	Owner + Accountant	☐
Backup internet connection at the counter, or paper bill book ready	Owner	☐
Agent running as a service on the Tally server; alerts tested	Integration	☐
New bill number series approved by the accountant under GST rules	Accountant	☐
Tally data quality sample checked and cleaned (duplicate parties, phone numbers, item names)	Accountant + PM	☐

15. Staff Handover and Training Manual

Written in simple language for the owner and shop staff. No technical knowledge needed.

15.1 What You Receive

- Website and back-office links, POS installed on each counter, and a logged-in test account for practice.
- A separate login for every staff member (never share logins).
- This documentation, the SRS, design files and source code access.
- Recorded training videos, a one-page quick guide per role, and 30 days of free support *[Assumption]*.

15.2 Daily Tasks

Until cutover (Mode A) the counter keeps billing in TallyPrime as today. The rows that say **POS** apply after the counter moves to the FashionOS POS.

Task	Who	Where	Steps
Make a cash bill	Cashier	POS	Scan each item, check the total, choose Cash, take money, press Done, print or send on WhatsApp
Make a khata (udhar) bill	Cashier	POS	Find the customer by phone, scan items, choose Khata, check balance and limit, press Done. The customer gets the bill on WhatsApp
Customer is over the limit	Manager	POS	The screen warns. Only a manager or owner can allow it, with a reason
Take a khata payment in the shop	Cashier	Khata → customer → Receive payment	Enter amount and mode, save. The balance and Tally update by themselves
Customer says they paid from home	Cashier	Khata → customer → Ledger	Check the payment in the ledger; if missing, wait 10 minutes, then call support with the customer's phone
Return an item	Cashier	POS → Returns	Find the original bill, choose items, choose refund or credit note, manager approves
Close the day	Cashier / Manager	POS → Cash register → Close	Count cash, enter it, check the difference, close. Bills are then locked
Mark attendance	Manager	People → Attendance	Mark each person Present, Half day, Leave or Absent before the day ends
Give an advance	Owner	People → employee → Give advance	Enter amount, reason, monthly deduction. The balance shows at once
Pay salary	Owner / Accountant	People → Salary	Check the draft, approve, record how it was paid, share the payslip
Create and track fall-pico	Cashier, Supervisor	POS, Fall-pico	Bill the saree, answer Yes to fall-pico, stick the printed tag, give to the supervisor to assign. When ready, the customer is told on WhatsApp
Pay a fall-pico worker	Supervisor / Owner	Fall-pico → Workers	Check completed and approved jobs, record payment, any advance is deducted
Receive new stock	Staff	Purchasing → New purchase	Enter the supplier bill, check quantities, print labels, stick labels
Pack an online order	Online order staff	Orders	Print pick list, scan each item, wait for fall-pico if shown, pack, create courier shipment
Check Tally problems	Accountant	Tally → Sync issues	Open each issue, fix the cause (for example a missing ledger), press Retry
Make a festival poster	Manager	AI tools → Poster maker (Phase 2)	Type the offer, choose photos, check the text, download

15.3 Things to Remember

- Never delete or change a khata entry. If something is wrong, make a correction entry with a reason.
- Always scan the barcode; do not type item names unless the label is missing.
- Check AI-written product text and posters before publishing.
- Do not share logins or PINs. Salary details are private.
- If the internet is off, keep billing. The POS will send bills later; do not make the same bill twice.
- Do not delete products that have past sales; mark them Inactive.
- Do not close Tally or the Tally server session. If Tally must be restarted, tell the accountant; entries wait safely and go through when it is open again.

15.4 Frequently Asked Questions

The POS shows Offline. Keep billing. Check the sync number at the corner; it should reach 0 when the internet returns.

A customer did not get the WhatsApp bill. Open the ledger entry and look at the status. Press Resend. If it still fails, the number may not be on WhatsApp; an SMS is sent instead.

Stock on the website is different from the shop. Check the stock history for that item, then do a stock count and adjust with a reason.

How do I change the fall-pico price limit or the return days? Settings → Rules (owner only).

How do I add a new staff member? Settings → Users → Add, then choose a role.

15.5 Support and Maintenance

Item	Detail
Support contact	TBD (phone, WhatsApp, email)
Support hours	TBD
Critical issue (billing down, wrong money)	Response within 2 hours <i>[Assumption]</i>
Normal issue	Within 1 working day
New features	Through the Change Request process (section 15.3)

15.6 Training Plan

Group	Time	Content
Cashiers	1 hour + practice day	Cash bill, khata bill, payments, returns, offline mode, day close
Manager and supervisor	2 hours	Discounts, approvals, attendance, fall-pico board, QC
Accountant	Half day	Tally sync, issues screen, reconciliation, GST, salary sheets
Owner	2 hours	Dashboard, approvals, ledgers, salary, reports
Online order staff	1 hour	Pick, pack, ship, returns

16. Risks, Assumptions and Change Log

16.1 Risks

#	Risk	Impact	Likelihood	Mitigation	Owner
R1	Tally sync is harder than expected (version, locked periods, duplicates)	Wrong books, delay	High	Check in Sprint 0; test company; outbox with unique references; daily reconciliation	Integration
R2	WhatsApp template approval is slow or rejected	Bills cannot be sent	Medium	Submit in Week 1; keep SMS fallback; simple template wording	Integration
R3	Scope is too large for 5 people	Late launch	High	Phase plan; shop first, website last; AI after go-live; strict change control	PM
R4	Offline POS creates conflicts or duplicate bills	Wrong stock or money	Medium	Idempotency keys, per-counter series, conflict alerts, heavy offline testing	Frontend + Backend

#	Risk	Impact	Likelihood	Mitigation	Owner
R5	Wrong opening balances at migration	Customer disputes	Medium	Owner sign-off per balance; parallel run	PM + Owner
R6	Staff resist the new system	Low adoption	Medium	Real cashier in design tests, training, paper bill backup	PM
R7	PM, QA and design on one person	Quality gaps	High	Part-time designer; QA support from developers	PM
R8	AI writes a wrong amount or unsafe text	Trust and money	Low	Placeholders, validator, standard template fallback, zero-error test	AI
R9	Legal gaps (payroll rules, khata interest, WhatsApp opt-in)	Fines, complaints	Medium	CA and lawyer review before launch	Owner
R10	Hardware problems (printers, scanners)	Slow billing	Medium	Test chosen models in Sprint 1; keep spare scanner	PM
R11	Salary and khata data exposed	Privacy and trust	Low	Role checks, encryption, access logs	Backend
R12	WhatsApp and AI costs higher than planned	Budget	Medium	Cost tracking, monthly caps, alerts	AI + PM

Added risks (v1.0):

#	Risk	Impact	Likelihood	Mitigation	Owner
R13	Counter billing in cloud Tally stops when the internet fails (Mode A)	Billing stops	Medium	Backup internet, paper bill book, move to the offline POS	PM + Owner
R14	Cloud Tally licence or hosting terms do not suit several users	Delay or extra cost	Medium	Check with the Tally vendor in Sprint 0	Owner + Accountant
R15	Tally data quality (duplicate parties, missing phone numbers) breaks WhatsApp bills	Wrong or missing messages	Medium	Sample check in Sprint 0; clean up before messaging; Sync Issues screen	PM + Accountant
R16	Tally server down or Tally not open, agent cannot post	Sync delay	Medium	Queue, auto-start, alerts, daily reconciliation	Integration
R17	New bill number series is wrong under GST	Compliance	Low	Design with the accountant; test on staging; rehearse cutover	Accountant
R18	Plain React site is weak for SEO	Low organic traffic	Medium	Decide prerender or server rendering before the website sprint	Frontend

16.2 Key Assumptions

- The open questions in SRS section 11 are answered by the end of Week 2.
- Tally runs on a cloud server in India that is on 24 hours (client decision; hosting and licence to confirm).
- The client supplies product data, opening balances, staff records and approvals on time.
- The accountant is available for Tally mapping, tests and the parallel run.
- Razorpay, WhatsApp provider, courier and AI vendor stay available at agreed prices.

16.3 Change Request Process

1. Client or team raises the request in writing.
2. PM logs it below with the impact on time and cost, and the SRS requirements it touches.

3. Client approves or rejects.
4. If approved: update the SRS first, then this document, then the task board.

16.4 Change Request Log

CR #	Date	Requested by	Description	Impact (days / cost)	Decision	Docs updated
CR-001						
CR-002						

16.5 Document Version History

Version	Date	Author	Changes
0.1	5 Oct 2026	Draft	First draft based on a marketplace / SaaS idea
0.2	5 Oct 2026	Draft	Rewritten for the client's own business (shop + website, shared stock) to match SRS v0.2: POS, khata, WhatsApp bills, Tally, employees, fall-pico, purchasing; new architecture, database, API, sprint plan, tests and go-live
1.0	5 Oct 2026	Draft	Stack decided (FastAPI, React + Vite, PostgreSQL, Celery + Redis); Tally on a 24-hour cloud server with the agent on the same server; Mode A first and POS cutover later; new roadmap, milestones, sprint plan and rollout order; new bill number series; one counter; added tests T34 to T41 and risks R13 to R18; matches SRS v1.0